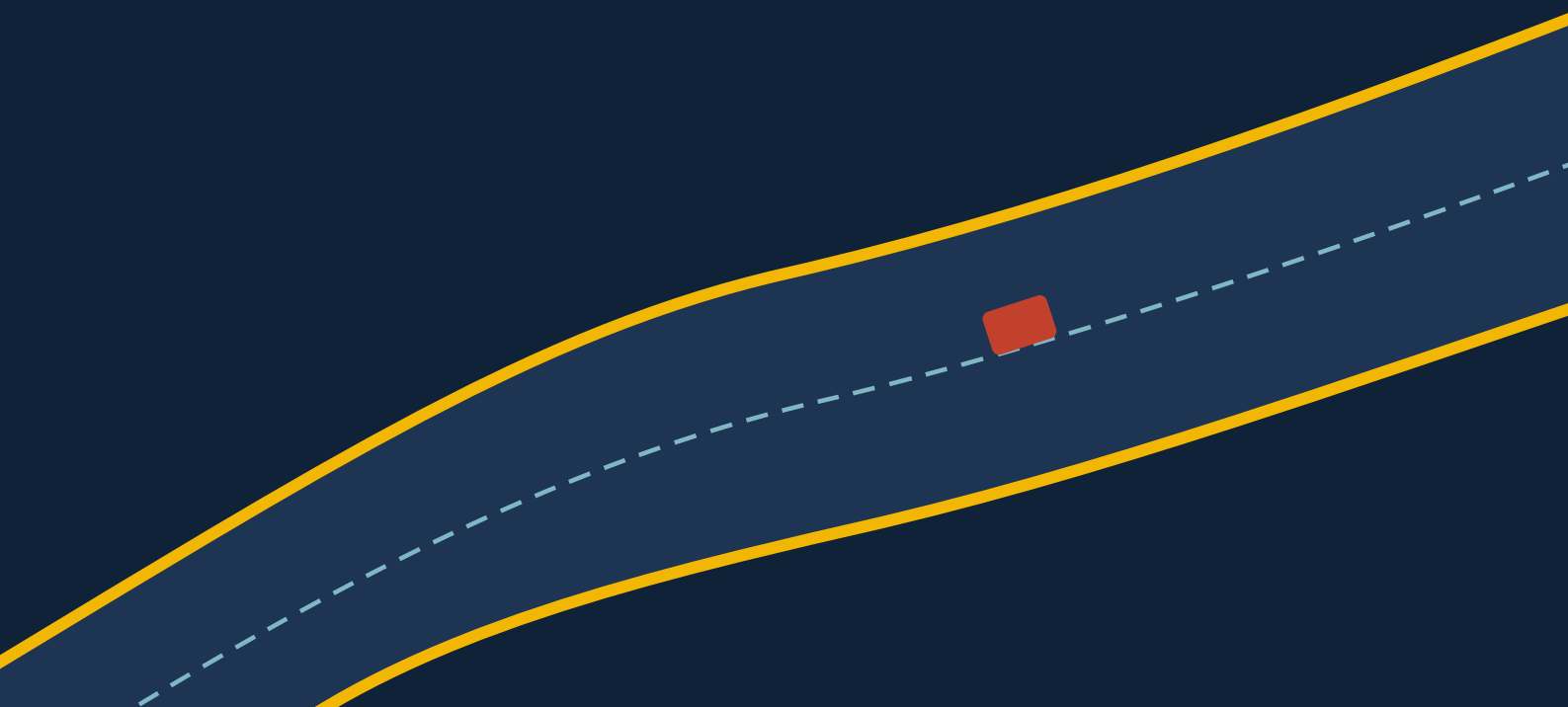
TÀI LIỆU ĐỊNH HƯỚNG KỸ THUẬT

# Nhận thức vật cản & Hoạch định hành lang an toàn cho xe tự hành tỉ lệ 1/7

Lộ trình phát triển tiếp từ hệ thống hiện có (Lane Detection V2 · LiDAR + Camera · NMPC · ROS 2) lên mức **Obstacle-aware Safe Corridor Planning**: xe tự nhận biết vật cản quanh mình, tự tạo hành lang an toàn và bám theo nó bằng NMPC.

LiDAR clustering & tracking | Frenet frame | Safe corridor | NMPC + ràng buộc hành lang | Behavior FSM | Safety monitor

Phiên bản 1.0 · 14/10/2026 | **Đối tượng:** nhóm phát triển xe tự hành 1/7 trên ROS 2



## 1 Tóm tắt định hướng

Hệ thống hiện tại đã giải xong bài toán **"đi trong làn"**: camera trích biên làn, fitting đa thức và tính CTE, sai lệch hướng, độ cong cho NMPC bám tâm làn, cả khi chuyển làn đơn trên sa bàn. Bước tiếp theo bạn muốn là **"đi trong khoảng trống an toàn"**: xe phải biết *phần nào của mặt đường đang trống* và tự chọn đường đi qua đó.

> **Ý tưởng cốt lõi, gói trong một câu**
> Dùng đường tâm làn (từ Lane Detection V2) làm trục tham chiếu, dựng **hệ tọa độ Frenet (s, d)**, chiếu vật cản LiDAR vào hệ này, rồi tại mỗi vị trí *s* phía trước tính khoảng ngang còn trống **$[d_{min}(s), d_{max}(s)]$**. Dải các khoảng này chính là **hành lang an toàn**, và NMPC chỉ cần thêm một ràng buộc: **$d_{min}(s_k) \le d_k \le d_{max}(s_k)$**.

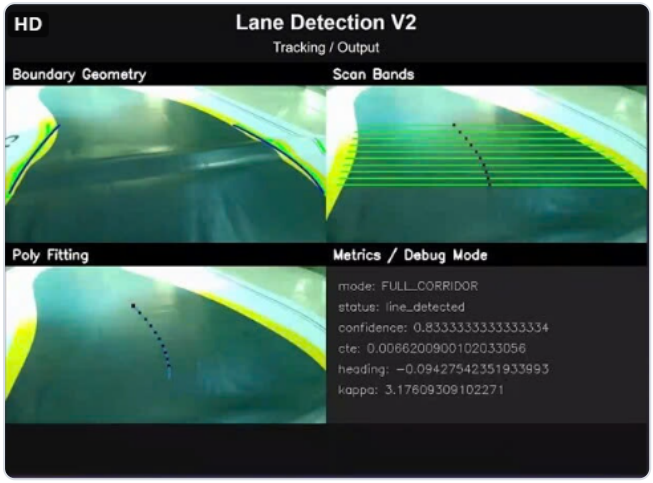
Hướng này được chọn vì nó **tận dụng tối đa những gì đã có**: lane detection vẫn giữ nguyên vai trò, NMPC vẫn giữ nguyên mô hình, chỉ thêm 3 khối mới (nhận diện vật cản, sinh hành lang, máy trạng thái hành vi) và một lớp an toàn độc lập. Không cần SLAM hay bản đồ toàn cục ở giai đoạn đầu.

### Các khối cần xây thêm

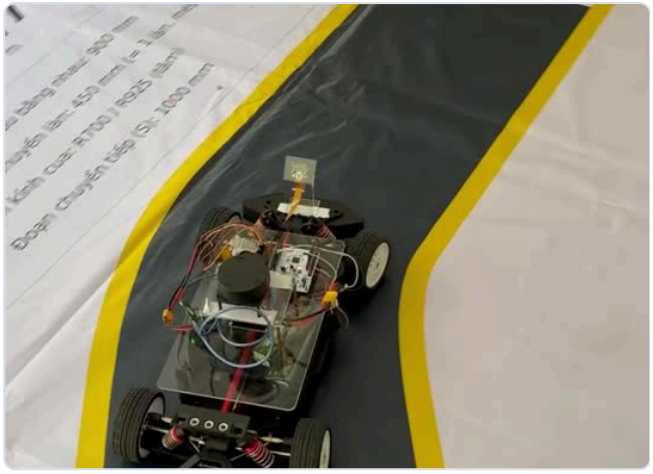
| Khối | Việc phải làm | Ưu tiên |
|---|---|---|
| **A. Nhận diện vật cản LiDAR** | Lọc vùng quan tâm → phân cụm theo breakpoint → fit hộp/ đường tròn → bám vết bằng Kalman filter, ước lượng vận tốc. | Bắt buộc |
| **B. Mô hình môi trường Frenet** | Tham số hóa đường tâm làn theo độ dài cung; chiếu vật cản và biên làn sang (s, d). | Bắt buộc |
| **C. Sinh hành lang an toàn** | Trừ "vùng cấm" của vật cản (đã phồng theo kích thước xe) ra khỏi khoảng trống của làn; chọn phía vượt bằng quy hoạch động. | Bắt buộc |
| **D. NMPC có ràng buộc hành lang** | Thêm ràng buộc mềm theo từng bước dự báo, mô hình hóa thân xe bằng các đường tròn. | Bắt buộc |
| **E. Máy trạng thái hành vi** | LANE_FOLLOW / AVOID / FOLLOW / STOP / E-STOP, có trễ chuyển (hysteresis). | Nên có |
| **F. Safety monitor** | Node độc lập đọc thẳng `/scan`, tự cắt `/actuation/arm` khi vật cản lọt vào vùng phanh. | Bắt buộc |
| **G. Camera cho vật cản** | Phát hiện vật thấp mà LiDAR 2D không quét tới; phân loại vật cản; kết hợp với LiDAR. | Mở rộng |

Ước lượng tổng thời gian: khoảng 11–12 tuần cho 1–2 người làm, chia 6 giai đoạn có tiêu chí nghiệm thu rõ ràng (mục 10).

## 2 Hiện trạng hệ thống & khoảng trống cần lấp




Lane Detection V2: biên làn, scan bands, poly fitting và các chỉ số (mode FULL_CORRIDOR, confidence, cte, heading, kappa).



Thử nghiệm chuyển làn đơn trên sa bàn 1/7 (cua R700/R925, offset chuyển làn 450 mm theo bảng thông số trên sa bàn).

### Những gì đã làm tốt

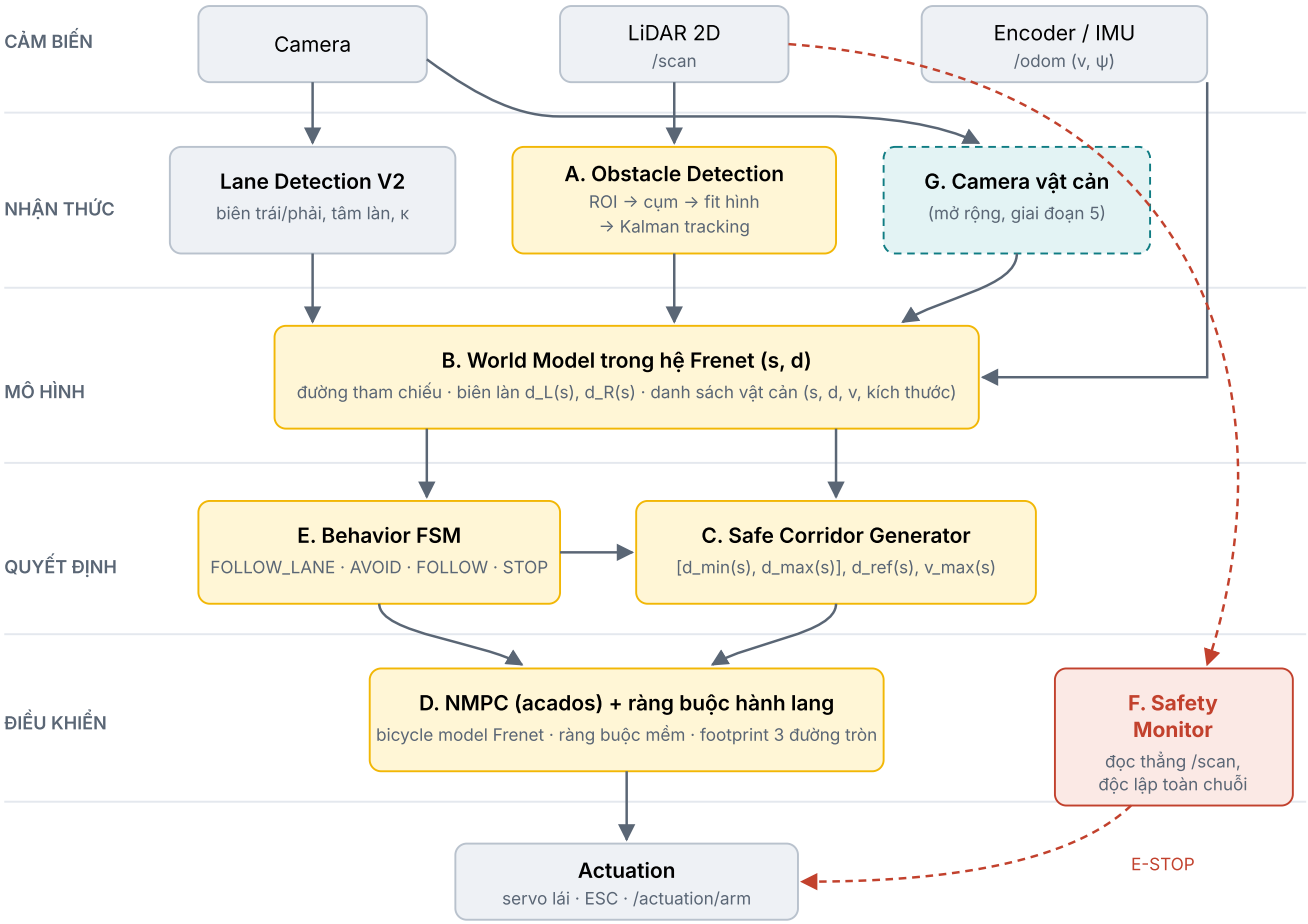
- Pipeline lane detection có cấu trúc rõ ràng (hình học biên → dải quét → fitting đa thức), có **chỉ số tin cậy** và chế độ (FULL_CORRIDOR), đây là nền rất tốt để mở rộng.
- Đầu ra đã ở dạng hình học mà bộ điều khiển dùng được ngay: **cte, sai lệch hướng, độ cong κ**.
- Đã chạy thật trên ROS 2 với NMPC, có dịch vụ bật/tắt chấp hành `/actuation/arm`, đã thử chuyển làn trên xe thật.
- Đã gắn LiDAR trên xe (thấy trong video), phần cứng cho bước tiếp theo đã sẵn sàng.

### Khoảng trống so với mục tiêu "hành lang an toàn"

| Hiện tại | Vấn đề khi có vật cản | Cần bổ sung |
|---|---|---|
| Lane chỉ cho ra **một đường tham chiếu** (tâm làn) | NMPC chỉ biết bám tâm, không có khái niệm "vùng được phép đi". | Biểu diễn **khoảng trống** [$d_{min}$, $d_{max}$] theo s |
| LiDAR chưa được khai thác cho vật cản | Xe "mù" với vật trên đường; camera nhìn gần, góc hẹp. | Khối A: phân cụm + tracking |
| Chuyển làn là kịch bản cố định | Không tự quyết định khi nào tránh, tránh bên nào, khi nào dừng. | Khối C + E: hành lang + FSM |
| Ràng buộc NMPC chủ yếu là giới hạn chấp hành | Không có đảm bảo khoảng cách tới vật cản. | Khối D: ràng buộc hành lang + thân xe |
| An toàn phụ thuộc vào cả chuỗi xử lý | Một node bị treo hoặc solver lỗi thì xe vẫn chạy. | Khối F: safety monitor độc lập |

## 3 Kiến trúc tổng thể đề xuất

Kiến trúc chia lớp theo kiểu **Sense → Perceive → Model → Decide → Plan → Control**, tương tự các stack lớn (Autoware, Apollo) nhưng rút gọn cho xe 1/7. Các khối màu vàng là phần mới, khối xám là phần đã có.




Hình 3.1: Kiến trúc tổng thể. Vàng: khối mới. Xám: đã có. Đỏ đứt nét: đường an toàn độc lập, đi tắt qua toàn bộ chuỗi lập kế hoạch.

**Nguyên tắc thiết kế**

- **Tách "hiểu môi trường" khỏi "điều khiển".** NMPC không cần biết vật cản là gì, nó chỉ nhận một hành lang. Nhờ vậy thay cảm biến hay thuật toán nhận thức sẽ không đụng tới bộ điều khiển.
- **Mọi thứ quy về hệ Frenet.** Trên đường đua có làn, (s, d) là hệ tự nhiên nhất: "tránh vật cản" chỉ còn là "giới hạn d".
- **An toàn không phụ thuộc vào thuật toán thông minh.** Safety monitor phải đơn giản tới mức có thể kiểm chứng bằng mắt.

# 4 Khối A: Nhận diện & bám vết vật cản bằng LiDAR

LiDAR nên là cảm biến chính cho vật cản vì nó cho **khoảng cách chính xác, không phụ thuộc ánh sáng**, và trên sa bàn phẳng thì một LiDAR 2D đã đủ. Pipeline đề xuất gồm 5 bước, tất cả chạy được dưới 5 ms trên máy tính nhúng với ~360–1000 điểm mỗi lần quét.

### A1. Tiền xử lý

- Biến đổi điểm từ `laser` sang `base_link` bằng TF tĩnh (phải đo chính xác vị trí và góc lắp LiDAR).
- **Cắt vùng quan tâm (ROI):** giữ điểm có 0.12 m < x < 2.5 m và $|y| < 1.0$ m (tùy tầm nhìn cần thiết); loại điểm rơi vào thân xe, dây và cột gắn camera.
- Loại điểm nằm **ngoài biên làn mở rộng** (dùng chính biên từ Lane V2 cộng biên độ 0.15 m) để không coi tường hay chân người đứng ngoài sa bàn là vật cản. Đây là chỗ camera và LiDAR bắt đầu "nói chuyện" với nhau.

### A2. Phân cụm theo breakpoint thích nghi

Với LiDAR 2D, các điểm đã có thứ tự theo góc, nên **không cần DBSCAN**. Cách nhanh hơn (O(n)) là duyệt tuần tự và tách cụm khi khoảng cách giữa hai điểm liên tiếp vượt ngưỡng tăng theo khoảng cách (Adaptive Breakpoint Detector, Borges & Aldon 2004):

$$\|p_i - p_{i-1}\| > D_{max} = r_{i-1} \cdot \sin(\Delta\varphi) / \sin(\lambda - \Delta\varphi) + 3\sigma_r$$

$\Delta\varphi$: độ phân giải góc · $\lambda$: góc tới hạn (≈ 10°) · $\sigma_r$: nhiễu đo khoảng cách. Loại cụm có < 3 điểm (nhiễu).

### A3. Ước lượng hình dạng

- **Vật nhỏ** (cọc, hộp < 10 cm): bao bằng **đường tròn** có tâm là trọng tâm cụm, bán kính là khoảng cách xa nhất.
- **Vật dài / mô hình xe khác:** dùng **L-shape fitting** (Zhang et al., 2017) để lấy hộp chữ nhật có hướng. Chú ý LiDAR chỉ thấy 1–2 mặt, nên kích thước thật phải dùng giá trị tối thiểu cấu hình trước hoặc cộng dồn qua nhiều khung hình.

### A4. Bám vết (tracking) và ước lượng vận tốc

- Mỗi vật cản là một track với **Kalman filter mô hình vận tốc hằng (CV)**, trạng thái $[x, y, v_x, v_y]$ trong hệ `odom` (không dùng `base_link`, nếu không vật đứng yên sẽ "chạy" ngược chiều xe).
- Ghép cụm với track bằng **thuật toán Hungarian** trên khoảng cách Mahalanobis, có cổng (gating) loại cặp quá xa.
- Quản lý vòng đời: track *tentative* → *confirmed* khi xuất hiện M/N khung (ví dụ 3/5) → *deleted* khi mất K khung. Việc này lọc rất tốt các vật cản "ma".
- Phân loại **tĩnh / động** theo $|v|$ có ngưỡng và trễ, vì khối C xử lý hai loại khác nhau.

### A5. Đầu ra

Message tự định nghĩa `ObstacleArray` (xem mục 9): id, tâm, kích thước, hướng, vận tốc, hiệp phương sai, cờ tĩnh/động, độ tin cậy. Đồng thời xuất `MarkerArray` để xem trên RViz2.

**Điểm mù của LiDAR 2D, cần biết từ đầu**

LiDAR 2D chỉ quét **một mặt phẳng ngang** ở độ cao lắp đặt. Vật cản **thấp hơn mặt quét** (vỉ, gờ, hộp dẹt) sẽ hoàn toàn vô hình. Ở các giai đoạn đầu, chọn vật cản thử nghiệm **cao hơn mặt quét ít nhất 3–5 cm**; vật thấp để dành cho Khối G (camera). Ngoài ra, vùng sát xe (< 10–15 cm) và vùng bị thân xe che cũng là điểm mù.

## 5 Khối B: Mô hình môi trường trong hệ Frenet

Hệ Frenet mô tả vị trí bằng **s** (quãng đường dọc theo đường tham chiếu) và **d** (độ lệch ngang, dương về bên trái). Trong hệ này, làn đường thẳng hay cong đều trở thành một "dải thẳng", và tránh vật cản trở thành bài toán 1 chiều theo d.

### B1. Đường tham chiếu

Có hai lựa chọn, nên làm lần lượt:

| Phương án | Cách làm | Ưu / nhược |
|---|---|---|
| **B1-a: Tham chiếu cục bộ** (làm trước) | Lấy đa thức tâm làn từ Lane V2 trong hệ `base_link`, lấy mẫu thành polyline, tham số hóa theo độ dài cung. Mỗi khung hình dựng lại. | + Không cần bản đồ<br>− Tầm nhìn ngắn (bằng tầm nhìn camera, ~1 m), đường có thể nhảy giữa các khung |
| **B1-b: Tham chiếu toàn cục** (nâng cấp) | Sa bàn cố định nên có thể dựng sẵn đường tâm (từ bản vẽ hoặc chạy 1 vòng ghi lại), định vị bằng odometry + đối khớp với làn đã thấy (hoặc AMCL / slam_toolbox với LiDAR). | + Tầm nhìn dài, hành lang ổn định, quan sát được vật cản sau khúc cua<br>− Cần định vị tốt |

**Mẹo quan trọng với B1-a**

Tầm nhìn camera ngắn hơn tầm LiDAR. Nên **ngoại suy đường tham chiếu** ra 2–2.5 m bằng cung tròn với độ cong $\kappa$ ước lượng (đã có sẵn trong Lane V2), và **làm mượt theo thời gian** (lọc hệ số đa thức bằng EMA hoặc Kalman) để hành lang không giật.

### B2. Chiếu vật cản sang (s, d)

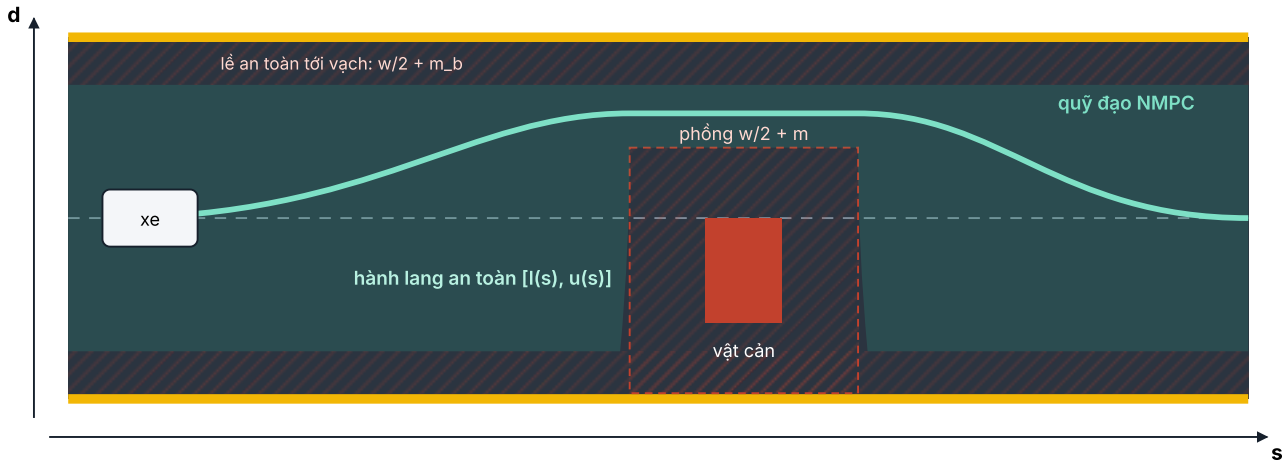
Với mỗi đỉnh của hộp (hoặc điểm trên đường tròn) của vật cản, tìm đoạn polyline gần nhất, chiếu vuông góc để lấy s, rồi lấy d bằng tích có hướng. Sau đó vật cản có dạng **khoảng** $[s_j^-, s_j^+] \times [d_j^-, d_j^+]$. Ở khúc cua gắt cần lấy mẫu dày trên chu vi vật, vì phép chiếu không tuyến tính.

### B3. Biên làn

Biên trái/phải từ Lane V2 cũng chiếu sang thành $d_L(s)$ và $d_R(s)$. Với sa bàn hiện tại, khi chỉ thấy một biên thì có thể dùng độ rộng làn đã biết để suy ra biên còn lại, đồng thời **hạ độ tin cậy** và thu hẹp hành lang.

# 6 Khối C: Sinh hành lang an toàn (trọng tâm)

Đây là khối trả lời đúng câu hỏi bạn đặt ra. Đầu ra là một dãy khoảng **$[l_i, u_i]$** tại các trạm $s_i$ phía trước. Nếu tâm xe (theo d) nằm trong dải đó ở mọi trạm thì xe không chạm biên làn và không chạm vật cản.




Hình 6.1: Hành lang an toàn (xanh) = vùng trong làn trừ lề vạch và trừ vùng cấm của vật cản (đỏ, đã phồng theo nửa bề rộng xe cộng lề m). Phía vượt được chọn là bên trái vì khoảng trống rộng hơn.

### Thuật toán (cho vật cản tĩnh)

1. **Rời rạc hóa:** $s_i = i \cdot \Delta s$, $i = 0..N$. Gợi ý $\Delta s = 0.05$ m, tầm nhìn $L = 1.5–2.5$ m (≈ 2–3 s ở tốc độ 0.8 m/s).
2. **Khoảng trống ban đầu từ làn:** $F_i = [d_R(s_i) + w/2 + m_b,\ \ d_L(s_i) - w/2 - m_b]$, với w là bề rộng xe, $m_b$ là lề tới vạch.
3. **Phồng vật cản:** vật j chiếm $[s_j^- - \ell_r - m_s,\ s_j^+ + \ell_f + m_s] \times [d_j^- - w/2 - m_o,\ d_j^+ + w/2 + m_o]$ ($\ell_f$, $\ell_r$: khoảng từ điểm tham chiếu tới mũi/đuôi xe; m: lề an toàn, có thể tăng theo độ bất định của track).
4. **Trừ khoảng:** tại mỗi $s_i$, lấy $F_i$ trừ đi các khoảng cấm. Kết quả có thể là 0, 1 hoặc nhiều khoảng con (ví dụ: vượt trái hoặc vượt phải).
5. **Chọn phía vượt (homotopy) bằng quy hoạch động:** mỗi khoảng con là một nút; nối nút ở trạm i với nút ở trạm i+1 nếu chúng chồng lấn đủ để xe chuyển qua, đồng thời thỏa giới hạn dịch ngang $|\Delta d| \le \Delta s \cdot \tan(\psi_{max})$. Chi phí nút = $-\alpha \cdot$(bề rộng) $+ \beta \cdot |$tâm $- d_{ref}|$ ; chi phí cạnh = $\gamma \cdot$(đổi phía). Chạy DP/Dijkstra lấy chuỗi có chi phí nhỏ nhất. Thêm chi phí **giữ quyết định của chu kỳ trước** để xe không lưỡng lự trái/phải.
6. **Hậu xử lý:** làm mượt biên theo s (không cho thu hẹp đột ngột quá khả năng bẻ lái), sinh $d_{ref}(s)$ = tâm hành lang đã làm mượt, và $v_{max}(s)$ giảm khi hành lang hẹp.
7. **Báo khả thi:** nếu tại trạm nào không còn khoảng nào, hoặc bề rộng < ngưỡng, thì báo `blocked_at_s` cho Behavior FSM để chuyển sang FOLLOW hoặc STOP.

### Mở rộng cho vật cản động

- **Cùng chiều, chậm hơn:** không tránh ngang mà xử lý theo trục dọc (giữ khoảng cách / ACC). Behavior FSM chọn FOLLOW và giới hạn v theo khoảng cách thời gian (time headway).
- **Cắt ngang / đi tới:** dự đoán vị trí theo mô hình CV ở các thời điểm $t_i \approx s_i / v_{ego}$ và đặt vùng cấm tại vị trí *dự đoán* thay vì vị trí hiện tại. Phồng thêm theo độ bất định tăng dần với t. Phiên bản chặt chẽ hơn

là đồ thị S–T (như Apollo EM Planner), nhưng chỉ nên làm khi bản tĩnh đã chạy ổn.

**Phương án nâng cao: hành lang lồi trong hệ Descartes**

Thay vì khoảng theo d, có thể biểu diễn hành lang là dãy **đa giác lồi** (Safe Flight Corridor, Liu et al. 2017), mỗi đa giác là tập ràng buộc tuyến tính $A_k p_k \le b_k$. Cách này tổng quát hơn (không cần làn), phù hợp nếu sau này muốn chạy ở bãi trống. Với sa bàn có làn, cách Frenet ở trên đơn giản và đủ tốt.

## 7 Khối D: Đưa hành lang vào NMPC

Bạn đã có NMPC chạy được, nên phần này chủ yếu là **thay đổi bài toán tối ưu**, không phải viết lại. Đề xuất dùng mô hình xe đạp động học trong hệ Frenet:

$$\dot{s} = v\cdot\cos(e_\psi) \,/\, (1 - \kappa_r(s)\cdot d)$$
$$\dot{d} = v\cdot\sin(e_\psi)$$
$$\dot{e}_\psi = v\cdot\tan(\delta)/L_{wb} - \kappa_r(s)\cdot\dot{s}$$
$$\dot{v} = a, \quad \dot{\delta} = \omega$$

Trạng thái $x = [s, d, e_\psi, v, \delta]$ · Điều khiển $u = [a, \omega]$ · $\kappa_r$: độ cong đường tham chiếu (đã có từ Lane V2) · $L_{wb}$: chiều dài cơ sở.

### Hàm mục tiêu

$$J = \Sigma_{k=0..N} \left[ q_d(d_k - d_{ref}(s_k))^2 + q_\psi e_{\psi,k}^2 + q_v(v_k - v_{ref,k})^2 + r_a a_k^2 + r_\omega \omega_k^2 + \rho_1 \varepsilon_k + \rho_2 \varepsilon_k^2 \right]$$

### Ràng buộc

- **Hành lang (ràng buộc mềm):** $l(s_k) - \varepsilon_k \le d_{c,k} \le u(s_k) + \varepsilon_k, \quad \varepsilon_k \ge 0$, với $\rho_1, \rho_2$ rất lớn. Ràng buộc mềm giúp solver **không bao giờ vô nghiệm** khi hành lang hẹp đột ngột. Thay vào đó $\varepsilon > 0$ là tín hiệu báo nguy hiểm cho lớp trên.
- **Thân xe bằng các đường tròn:** phủ thân xe bằng 2–3 đường tròn bán kính $r_c$ đặt tại độ lệch $\ell_c$ dọc thân. Tâm mỗi đường tròn có $d_c \approx d + \ell_c\cdot\sin(e_\psi)$. Khi dùng đường tròn thì phồng vật cản theo $r_c$ thay cho w/2. Chi tiết này quan trọng khi xe đang **xoay để chuyển làn**: mũi xe lệch xa hơn tâm.
- **Chấp hành:** $|\delta| \le \delta_{max}$, $|\omega| \le \omega_{max}$, $a_{min} \le a \le a_{max}$, $0 \le v \le v_{max}(s)$.
- **Gia tốc ngang:** $v^2\cdot|\tan\delta|/L_{wb} \le a_{y,max}$ (chống trượt ở cua R700).

**Chi tiết triển khai hay bị sai: tra hành lang theo s dự báo**

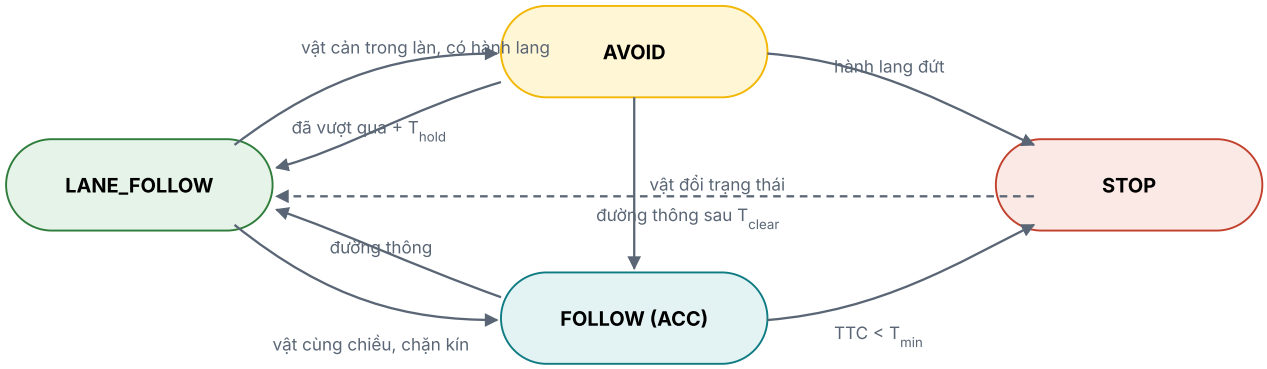
Hành lang được cho theo **s**, nhưng NMPC rời rạc theo **thời gian**. Mỗi chu kỳ, lấy quỹ đạo dự báo của lần giải trước (dịch đi 1 bước, warm start) để có $s_k$ dự kiến, tra ra $l_k$, $u_k$, $d_{ref,k}$ rồi nạp làm **tham số theo từng stage** (trong acados: `solver.set(k, "p", ...)` hoặc `constraints_set(k, "lh"/"uh")`). Nếu chỉ dùng một hành lang cho cả chân trời thì xe sẽ phanh hoặc né quá sớm.

### Solver & tham số gợi ý

| Hạng mục | Gợi ý ban đầu | Ghi chú |
|---|---|---|
| Thư viện | acados (SQP-RTI + HPIPM), sinh mã C từ Python/CasADi | Nhanh nhất cho nhúng; nếu đang dùng CasADi + IPOPT thì nên chuyển |
| Chu kỳ / chân trời | $T_s$ = 0.05 s, N = 20–30 (1.0–1.5 s) | Chân trời × $v_{max}$ không được vượt tầm hành lang |
| Thời gian giải | Mục tiêu p95 < 15 ms | Đo trên máy thật, ghi log mỗi chu kỳ |
| Dự phòng | Solver lỗi 3 chu kỳ liên tiếp hoặc max ε > ngưỡng → STOP | Không bao giờ gửi lệnh từ nghiệm lỗi |
| Bù trễ | Dự báo trạng thái tiến lên một khoảng bằng trễ đo được trước khi giải | Trễ camera → lệnh thường 60–120 ms, cần đo |

## 8 Khối E & F: Lớp hành vi và lớp an toàn

### E. Máy trạng thái hành vi




Hình 8.1: Máy trạng thái hành vi. Mọi chuyển trạng thái đều có thời gian giữ tối thiểu (hysteresis) để xe không "giật cục".

| Trạng thái | Đầu ra cho Corridor/NMPC |
|---|---|
| LANE_FOLLOW | Hành lang = toàn làn; $d_{ref}$ = 0; $v_{ref}$ = tốc độ hành trình. |
| AVOID | Hành lang đã trừ vật cản, phía vượt đã chốt; $d_{ref}$ = tâm hành lang; $v_{ref}$ giảm theo bề rộng hành lang. |
| FOLLOW | $d_{ref}$ = 0; $v_{ref} = \min(v_{cruise}, v_{obs} + k\cdot(gap - gap_{des}))$. |
| STOP | $v_{ref}$ = 0, giảm tốc êm với $a_{min}$ đã cấu hình; giữ lái theo làn. |

### F. Safety monitor: lưới an toàn cuối cùng

- Node **riêng, đơn giản, không phụ thuộc** perception/planner. Chỉ đọc `/scan` và `/odom`.

- Tính vùng phanh phía trước: $D_{brake} = v^2/(2 \cdot a_{brake}) + v \cdot t_{delay} + d_0$; vùng là hình chữ nhật rộng bằng xe + lề, dài $D_{brake}$.
- Có ≥ n điểm LiDAR trong vùng (lọc nhiễu bằng n ≥ 3) → gửi lệnh dừng với **ưu tiên cao nhất** (qua twist_mux hoặc gọi thẳng `/actuation/arm` = false, dịch vụ này bạn đã có).
- Watchdog: nếu không nhận lệnh điều khiển mới sau 200 ms (node treo) → dừng.

> **Vì sao tách riêng?**
> Khi thử nghiệm hành lang và NMPC, chắc chắn sẽ có lúc bug làm xe lao vào vật cản. Safety monitor giúp bạn **dám thử nhanh** mà không làm hỏng xe, và trong báo cáo đây là điểm cộng lớn về tính an toàn của hệ thống.

## 9 Tổ chức trên ROS 2

### Node & topic

| Node | Subscribe | Publish | Tần số |
|---|---|---|---|
| lane_detector (đã có) | /camera/image_raw | /perception/lane (LaneBoundaries) | ≥ 20 Hz |
| obstacle_detector | /scan, /perception/lane, TF | /perception/obstacles (ObstacleArray) | = tần số LiDAR |
| world_model_frenet | /perception/lane, /perception/obstacles, /odom | /planning/frenet_world | 20 Hz |
| behavior_fsm | /planning/frenet_world | /planning/behavior | 20 Hz |
| corridor_generator | /planning/frenet_world, /planning/behavior | /planning/corridor (Corridor) | 20 Hz |
| nmpc_controller (sửa) | /planning/corridor, /odom | /control/cmd | 20 Hz |
| safety_monitor | /scan, /odom, /control/cmd | /control/estop, gọi /actuation/arm | = tần số LiDAR |

### Message tự định nghĩa (gợi ý)

```
# Obstacle.msg
uint32 id
geometry_msgs/Pose pose      # odom
geometry_msgs/Vector3 size   # dài, rộng,
cao
geometry_msgs/Twist velocity
float32[4] pos_cov           # 2x2 x,y
bool is_dynamic
float32 confidence
uint8 shape                  # 0 tròn,1 hộp

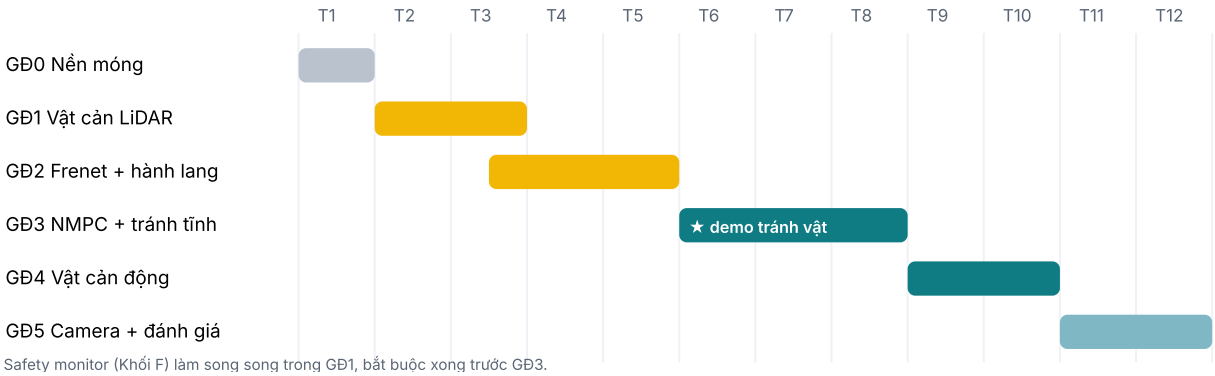
# ObstacleArray.msg
std_msgs/Header header
Obstacle[] obstacles
```

```
# Corridor.msg
std_msgs/Header header
float32 ds
float32[] s           # trạm s_i
float32[] lower       # l(s_i)
float32[] upper       # u(s_i)
float32[] d_ref
float32[] v_max
float32[] kappa_ref
uint8 side            # 0 giữa,1 trái,2
phải
bool feasible
float32 blocked_at_s
```

### Thực hành kỹ thuật nên áp dụng ngay

- **Cây TF chuẩn:** `odom → base_link → {laser, camera}`, đo và hiệu chuẩn ngoại tham số camera–LiDAR (đặt một vật có cạnh rõ, vừa thấy trên ảnh vừa thấy trên scan).
- **QoS:** `SensorDataQoS` (best effort, depth 1) cho cảm biến; `reliable` cho corridor và lệnh điều khiển.
- **Dấu thời gian:** mọi message mang header.stamp của cảm biến gốc để đo trễ đầu–cuối. Ghép dữ liệu bằng `message_filters` hoặc lấy mẫu gần nhất có kiểm tra tuổi dữ liệu.
- **Composable nodes** (cùng process, zero-copy) cho chuỗi perception → planner để giảm trễ.
- **RViz2 là bắt buộc:** vẽ vật cản, hành lang (hai đường biên + mặt tô), quỹ đạo dự báo của NMPC. Bạn đã có giao diện debug cho lane; mở rộng thêm một panel cho corridor.
- **rosbag2:** ghi lại mọi lần chạy. Khối B, C phát triển và kiểm thử **offline trên bag** trước khi chạy thật, nhanh hơn rất nhiều.

# 10 Lộ trình triển khai theo giai đoạn




Hình 10.1: Tiến độ dự kiến khoảng 12 tuần. GĐ1 và GĐ2 chồng nhau 0.5 tuần vì GĐ2 phát triển offline trên dữ liệu bag của GĐ1.

### GĐ0: Nền móng

1 tuần

Cây TF chuẩn, hiệu chuẩn camera–LiDAR, đo trễ từng khâu, script ghi bag. Chuẩn bị bộ vật cản thử nghiệm (hộp, cọc cao hơn mặt quét LiDAR). Ghi ≥ 20 bag có vật cản ở nhiều vị trí: giữa làn, lệch trái, lệch phải, trong cua.

**Nghiệm thu:** RViz2 hiển thị đúng scan chồng lên ảnh/biên làn; có bảng trễ (ms) của từng khâu.

### GĐ1: Nhận diện & bám vết vật cản LiDAR (Khối A + F)

2 tuần

Cài đặt A1–A5, viết safety monitor + watchdog.

**Nghiệm thu:** phát hiện ≥ 95% vật cản tĩnh trong 2 m; sai số tâm < 3 cm; ID giữ ổn định khi xe chạy; ≤ 1 vật cản "ma" mỗi phút; safety monitor dừng xe đúng trong 10/10 lần thử đẩy vật vào đầu xe.

### GĐ2: World model Frenet + Sinh hành lang (Khối B + C)

2.5 tuần

Làm hoàn toàn offline trên bag trước, rồi mới chạy online (xe vẫn chạy theo bộ điều khiển cũ, chỉ hiển thị hành lang).

**Nghiệm thu:** hành lang đúng bằng mắt trên 100% kịch bản trong bag; phía vượt không đổi qua lại; thời gian tính < 5 ms/chu kỳ.

### GĐ3: NMPC với ràng buộc hành lang + Behavior FSM (Khối D + E): mốc demo chính

3 tuần

Thêm ràng buộc mềm, footprint đường tròn, tham số theo stage; FSM 4 trạng thái. Bắt đầu ở 0.3 m/s, tăng dần.

**Nghiệm thu:** tránh thành công 1 vật tĩnh (≥ 9/10 lần), 2 vật so le (slalom), vật chắn kín làn → dừng êm trước vật; khoảng cách tối thiểu tới vật ≥ lề thiết kế; solver p95 < 15 ms.

**GĐ4: Vật cản động (dự đoán + ACC)** 2 tuần

Dùng xe thứ hai (hoặc vật kéo bằng dây) chạy chậm cùng chiều và cắt ngang; thêm dự đoán CV vào khối C, trạng thái FOLLOW.

**Nghiệm thu:** bám theo xe trước với khoảng cách ổn định (dao động < ±10 cm); dừng/nhường khi vật cắt ngang; không va chạm trong 20 lượt.

**GĐ5: Camera cho vật cản + đánh giá tổng hợp (Khối G)** 2 tuần

Xem mục 11. Chạy bộ kịch bản đầy đủ, tổng hợp số liệu, quay video, viết báo cáo.

**Nghiệm thu:** phát hiện được vật thấp mà LiDAR bỏ sót; bảng đánh giá đủ các chỉ số ở mục 12.

## 11 Khối G: Dùng camera cho vật cản (mở rộng)

Camera bổ sung cho LiDAR ở hai điểm: **vật thấp** (dưới mặt quét) và **ngữ nghĩa** (biết vật cản là gì để chọn hành vi). Có ba mức, từ rẻ tới đắt:

| Mức | Cách làm | Đánh giá |
|---|---|---|
| **G1. Bất thường mặt đường** (rất nên thử) | Mặt sa bàn màu tối đồng nhất. Sau phép biến đổi phối cảnh ngược (IPM, bird's-eye view) mà Lane V2 đã gần như có sẵn, mọi vùng pixel **bên trong làn nhưng khác màu mặt đường** là ứng viên vật cản. Lấy contour, chiếu ra mét. | + Gần như miễn phí, không cần huấn luyện<br>− Giả định đường phẳng: vật cao bị "kéo dài", nên chỉ dùng cạnh gần nhất của vật |
| **G2. Phát hiện đối tượng** | YOLO bản nano, huấn luyện trên ảnh tự chụp ở sa bàn (cọc, hộp, xe mô hình, hình nhân), xuất NCNN/TensorRT, INT8. | + Có lớp đối tượng<br>− Cần gán nhãn vài trăm ảnh; tốn tính toán trên máy nhúng |
| **G3. Kết hợp (late fusion)** | Chiếu cụm LiDAR lên ảnh bằng ngoại tham số đã hiệu chuẩn, ghép với box camera theo IoU. Vị trí lấy từ LiDAR, lớp lấy từ camera; box camera không có cụm LiDAR thì giữ lại làm "vật thấp". | + Bổ sung cho nhau<br>− Cần hiệu chuẩn và đồng bộ thời gian tốt |

**Khuyến nghị**

Làm **G1 trước**: nhanh, có ý nghĩa nghiên cứu (vì giải đúng điểm mù của LiDAR 2D) và không đè nặng máy nhúng. G2/G3 làm nếu còn thời gian hoặc nếu đề tài cần phần "AI" rõ ràng hơn.

## 12 Kịch bản thử nghiệm & chỉ số đánh giá

| Mã | Kịch bản | Kỳ vọng |
|---|---|---|
| S1 | Không vật cản, chạy hết vòng | Như hiện tại, hành lang không gây xáo trộn |

| | | |
|---|---|---|
| S2 | 1 vật tĩnh lệch phải trên đoạn thẳng | Né trái êm, quay về tâm làn |
| S3 | 1 vật tĩnh trong cua R700 | Né đúng phía, không lấn vạch |
| S4 | 2 vật so le (slalom) | Đổi phía 1 lần, không lưỡng lự |
| S5 | Vật chắn kín làn | STOP êm trước vật, khoảng cách ≥ $d_0$ |
| S6 | Vật xuất hiện đột ngột ở gần | Safety monitor dừng xe |
| S7 | Xe trước chạy chậm cùng chiều | FOLLOW, giữ khoảng cách |
| S8 | Vật thấp dưới mặt quét LiDAR | Chỉ phát hiện được khi có G1/G3 |

| Chỉ số | Định nghĩa / cách đo |
|---|---|
| Tỉ lệ thành công | Số lượt hoàn thành không chạm vật/vạch ÷ tổng lượt (≥ 10 lượt mỗi kịch bản) |
| Khoảng cách tối thiểu tới vật | min khoảng cách footprint – vật, tính từ log LiDAR |
| CTE RMS / max | Khi không có vật cản: so với kết quả hiện tại để chứng minh không bị kém đi |
| Độ êm | max $\|\delta\|$, max gia tốc ngang, jerk |
| Thời gian giải NMPC | trung bình, p95, max; tỉ lệ $\varepsilon > 0$ |
| Trễ đầu–cuối | stamp cảm biến → lệnh tới chấp hành |
| Dừng nhầm / bỏ sót | Số lần dừng khi đường thông; số vật cản không được phát hiện |

## 13 Rủi ro thường gặp & cách phòng

| Rủi ro | Biểu hiện | Cách phòng |
|---|---|---|
| Đường tham chiếu nhảy giữa các khung | Hành lang rung, xe lắc | Lọc theo thời gian; neo tham chiếu trong `odom` thay vì `base_link`; chuyển sang B1-b |
| Lưỡng lự trái/phải | Xe lắc qua lại trước vật cản | Chi phí giữ quyết định trong DP + hysteresis FSM |
| Solver vô nghiệm / chậm | Lệnh mất, xe giật | Ràng buộc mềm, warm start, giới hạn số vòng lặp SQP, fallback STOP |
| Trễ lớn | Né muộn, dao động | Đo trễ thật; dự báo trạng thái bù trễ; composable nodes |
| Phản xạ LiDAR | Vạch, băng keo phản quang, sàn bóng tạo điểm giả | Lọc theo cường độ, M/N confirm, ROI theo làn |

| | | |
|---|---|---|
| Quá tải CPU | Tần số tụt khi chạy cả YOLO | Làm G1 trước G2; đo tải CPU từng node; giảm độ phân giải ảnh |
| Hỏng phần cứng khi thử | Xe đâm vật / rơi khỏi sa bàn | Safety monitor xong trước GĐ3; vật cản mềm (xốp); tăng tốc dần |

## 14 Thông tin cần làm rõ để tinh chỉnh lộ trình

1. Model LiDAR (tần số quét, độ phân giải góc, tầm gần nhất) và **độ cao lắp đặt** so với mặt sa bàn.
2. Máy tính nhúng đang dùng (Raspberry Pi 4/5, Jetson...), tải CPU hiện tại khi chạy lane + NMPC.
3. NMPC đang viết bằng gì (CasADi/IPOPT, acados, tự viết), mô hình, N, $T_s$, thời gian giải hiện tại.
4. Đầu ra lane đang ở hệ tọa độ nào (pixel hay mét sau IPM), đã có odometry từ encoder/IMU chưa.
5. Mục tiêu cuối: báo cáo đồ án, bài báo, hay cuộc thi? Tiêu chí chấm sẽ quyết định nên đầu tư sâu vào phần nào (an toàn, AI camera, hay điều khiển).

## 15 Tài liệu tham khảo chính


1. M. Werling, J. Ziegler, S. Kammel, S. Thrun, "Optimal Trajectory Generation for Dynamic Street Scenarios in a Frenét Frame," *IEEE ICRA*, 2010. Nền tảng của hệ Frenet trong lập kế hoạch.
2. S. Liu et al., "Planning Dynamically Feasible Trajectories for Quadrotors Using Safe Flight Corridors in 3-D Complex Environments," *IEEE Robotics and Automation Letters*, 2017. Khái niệm hành lang an toàn lồi.
3. H. Fan et al., "Baidu Apollo EM Motion Planner," arXiv:1807.08048, 2018. Tách quy hoạch đường (SL) và tốc độ (ST), cách dựng biên d theo s.
4. A. Liniger, A. Domahidi, M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," *Optimal Control Applications and Methods*, 2015. MPC cho xe tỉ lệ nhỏ.
5. R. Verschueren et al., "acados: a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, 2022.
6. G. A. Borges, M.-J. Aldon, "Line extraction in 2D range images for mobile robotics," *Journal of Intelligent and Robotic Systems*, 2004. Adaptive breakpoint detector.
7. X. Zhang, W. Xu, C. Dong, J. M. Dolan, "Efficient L-Shape Fitting for Vehicle Detection Using Laser Scanners," *IEEE Intelligent Vehicles Symposium*, 2017.
8. S. Macenski et al., "The Marathon 2: A Navigation System," *IEEE/RSJ IROS*, 2020. Nav2, tham khảo cách tổ chức costmap và an toàn trên ROS 2.
9. F1TENTH Autonomous Racing, tài liệu khóa học và mã nguồn mở (f1tenth.org). Rất sát quy mô xe 1/7–1/10.